# Guía: de los Transformers a la GPU

*Basada en tu resumen «Transformers y GPU» y en el video de NVIDIA. Orden: Transformers → historia y funcionamiento de la GPU → hardware → ejemplo de entrenamiento en tablero → código creativo.*

## 1. Transformers: la base que da paso a la GPU

### 1.1 El problema que resolvieron

Los Transformers son la arquitectura detrás de ChatGPT, Claude, Gemini, BERT y muchos modelos de imagen y audio. Se presentaron en 2017 en el paper *Attention Is All You Need* (Google).

Antes se usaban redes recurrentes (**RNN y LSTM**), que tenían dos problemas:

- **Procesaban palabra por palabra, en orden.** Cada paso dependía del anterior, así que no se podía paralelizar y entrenar era lento.
- **Olvidaban el contexto lejano.** En frases largas, lo del inicio se diluía al llegar al final.

El Transformer elimina la recurrencia: procesa **toda la secuencia a la vez** y deja que cada palabra mire directamente a todas las demás.

### 1.2 La idea central: self-attention

Cada palabra (convertida en vector) genera tres versiones de sí misma:

- **Q (Query):** lo que busco.
- **K (Key):** lo que ofrezco.
- **V (Value):** la información que aporto.

```
Atención(Q, K, V) = softmax( Q · Kᵀ / √d_k ) · V
```

Paso a paso: (1) producto punto de Q con todas las K para obtener puntajes, (2) dividir entre √d_k (d_k = tamaño del vector K) para que no crezcan demasiado, (3) softmax para convertirlos en porcentajes que suman 100 %, (4) mezclar los V según esos porcentajes.

**Ejemplo con «El gato duerme»:** al procesar «duerme», el softmax da 4,5 % a «El», 76,8 % a «gato» y 18,7 % a «duerme». El nuevo vector de «duerme» queda compuesto en gran parte por información de «gato»: la palabra entendió quién es el que duerme.

### 1.3 Componentes de un Transformer

| Componente | Qué hace |
|---|---|
| Tokenización y embeddings | Convierte texto en números y luego en vectores con significado |
| Codificación posicional | Suma la posición de cada palabra, porque todo se procesa a la vez |
| Atención multi-cabeza | Varias atenciones en paralelo, cada una aprende un tipo de relación |
| Red feed-forward | Capa densa con activación (GELU o ReLU) aplicada a cada token |
| Residuales + Layer Norm | Atajos y normalización para apilar muchas capas sin que el entrenamiento falle |

Un bloque es *Atención → Feed-forward*, y se apilan decenas de ellos (GPT-3 tiene 96).

### 1.4 Por qué se paralelizan: la transición a la GPU

- La tabla de atención de una frase de $n$ tokens tiene $n \times n$ puntajes. Con 1000 tokens son un millón, repetidos en cada capa y en cada cabeza.
- Cada fila de esa tabla se calcula sin depender de las demás.
- Todo se reduce a **multiplicaciones de matrices**.

> «Los Transformers cambiaron la IA porque pueden calcularse en paralelo. Pero ese paralelismo exige un hardware capaz de hacer millones de multiplicaciones de matrices a la vez: la GPU.»

## 2. Historia de la GPU: de dibujar píxeles a multiplicar matrices

### 2.1 Línea de tiempo

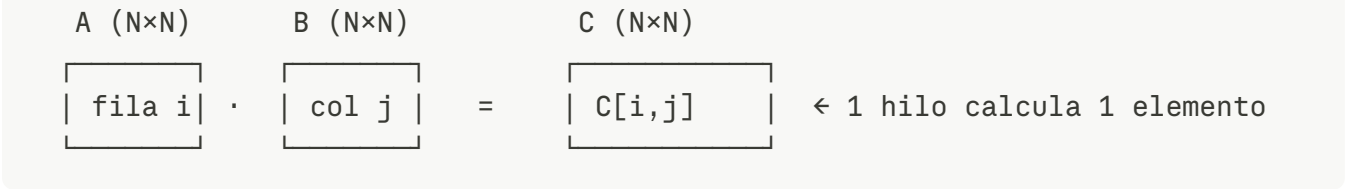
| Año | Hito | Relevancia para las matrices |
|---|---|---|
| 1999 | GeForce 256, primera «GPU» | Hecha para gráficos de videojuegos: transformar miles de vértices y píxeles en paralelo |

| Año | Hito | Relevancia para las matrices |
|---|---|---|
| 2001 | Shaders programables (GeForce 3) | Por primera vez se podía programar qué hace la tarjeta |
| 2006 | Arquitectura Tesla (G80) | Procesadores unificados: núcleos de propósito general |
| 2007 | **CUDA** | Se programa la GPU en C/C++ para cálculo general, no solo gráficos |
| 2012 | **AlexNet** con 2 GPUs | Demuestra que entrenar redes profundas en GPU es viable y gana ImageNet |
| 2016 | Pascal + **NVLink** | Conexión rápida entre GPUs |
| 2017 | Volta con **Tensor Cores** | Hardware dedicado a multiplicar matrices |
| 2018 | Primer **NVSwitch** (DGX-2) | Cualquier GPU puede hablar con cualquier otra |
| 2020 | Ampere | Tensor Cores más flexibles (TF32, BF16) |
| 2022 | Hopper | Transformer Engine con precisión FP8, pensado para Transformers |
| 2024 | Blackwell | Más rendimiento y precisiones aún menores (FP4) |

**Idea para contar:** los gráficos ya eran, en el fondo, multiplicaciones de matrices (rotar, escalar y proyectar objetos 3D). La GPU se hizo experta en eso por los videojuegos, y la IA aprovechó esa habilidad.

## 2.2 Cómo multiplica matrices una GPU

Para C = A · B, cada elemento C[i, j] es el producto punto de la fila *i* de A con la columna *j* de B. **Ningún elemento depende de otro**, así que se pueden calcular todos a la vez.




- **Hilos (threads):** cada uno calcula uno o pocos elementos de C.
- **Warps:** grupos de 32 hilos que ejecutan la misma instrucción a la vez (modelo SIMT).
- **Bloques y grid:** los hilos se agrupan en bloques, los bloques forman una cuadrícula (grid) y cada bloque corre en un multiprocesador (SM). Una GPU moderna tiene más de cien SMs.

- **Tiling (mosaicos):** para no leer una y otra vez la memoria lenta, cada bloque copia un pedazo de A y B a la **memoria compartida** (rápida) y trabaja ahí.
- **Tensor Cores:** circuitos que multiplican bloques pequeños de matrices (por ejemplo 4×4) en un solo paso, con precisión reducida (FP16, BF16, FP8) para ir más rápido y gastar menos memoria.

## 2.3 Cómo hacer estas operaciones: herramientas, comandos y código

| Nivel | Herramienta | Para qué sirve |
|---|---|---|
| Alto | **PyTorch**, TensorFlow/Keras, JAX | Escribes `A @ B` y el framework usa la GPU |
| Arrays | **CuPy** | Como NumPy, pero en GPU |
| Librerías | **cuBLAS** (matrices), **cuDNN** (redes), **CUTLASS** | Multiplicaciones ultra optimizadas |
| Kernels propios | **CUDA C++**, **Triton**, Numba | Escribir tu propio programa para la GPU |
| Multi-GPU | **NCCL**, `torchrun` | Comunicar y repartir trabajo entre GPUs |
| Diagnóstico | `nvidia-smi`, Nsight | Ver uso, memoria y temperatura |

### Comandos básicos

```
nvidia-smi                # ¿hay GPU? modelo, memoria, uso
nvcc --version            # versión del compilador CUDA
pip install torch         # instala PyTorch (en pytorch.org eliges la
versión con CUDA)
python -c "import torch; print(torch.cuda.is_available())"
```

Si no tienen GPU, **Google Colab** da una gratis: *Entorno de ejecución → Cambiar tipo de entorno → GPU (T4)*, y en una celda `!nvidia-smi`.

### Nivel 1: la misma multiplicación en CPU y en GPU con PyTorch

```
import torch, time

N = 8192
for dev in ['cpu', 'cuda']:
    A = torch.randn(N, N, device=dev)
    B = torch.randn(N, N, device=dev)
    A @ B                                   # calentamiento
    if dev == 'cuda': torch.cuda.synchronize()
    t = time.perf_counter()
    C = A @ B                               # ~1.1 billones de operaciones
    if dev == 'cuda': torch.cuda.synchronize()
    print(dev, round(time.perf_counter() - t, 3), 's')
```

`synchronize()` es necesario porque la GPU trabaja de forma asíncrona: sin él, el cronómetro se detendría antes de que termine el cálculo.

**Nivel 2: usar los Tensor Cores con precisión mixta**

```
with torch.autocast('cuda', dtype=torch.float16):
    C = A @ B        # ahora corre en Tensor Cores
```

**Nivel 3: un kernel CUDA propio (un hilo por elemento de C)**

```
__global__ void matmul(const float* A, const float* B, float* C, int N) {
    int fila = blockIdx.y * blockDim.y + threadIdx.y;
    int col  = blockIdx.x * blockDim.x + threadIdx.x;
    if (fila < N && col < N) {
        float suma = 0.0f;
        for (int k = 0; k < N; k++)
            suma += A[fila * N + k] * B[k * N + col];
        C[fila * N + col] = suma;
    }
}
// Lanzamiento:
// dim3 hilos(16, 16);
// dim3 bloques((N + 15) / 16, (N + 15) / 16);
// matmul<<<bloques, hilos>>>(d_A, d_B, d_C, N);
```

Este kernel es didáctico (lee mucho la memoria lenta). cuBLAS hace lo mismo con tiling y Tensor Cores y es mucho más rápido, por eso en la práctica se usa la librería.

## 3. Hardware de NVIDIA: más que una GPU

Según el video, NVIDIA no quiso fabricar solo el cerebro de las redes neuronales, sino un cuerpo completo:

| Parte del cuerpo | Qué es en el hardware |
|---|---|
| **Cerebro** | La GPU (arquitectura Blackwell) |
| **Sistema nervioso** | Las redes que conectan todo: NVLink, NVSwitch, Spectrum-X |
| **Sistema inmunológico** | La DPU BlueField-3: seguridad y descarga de tareas de red |
| **Esqueleto** | El rack y la integración física (GB200 NVL72) |

## Por qué no basta Ethernet convencional

Entrenar un modelo reparte el trabajo entre miles de GPUs, que deben intercambiar resultados todas al mismo tiempo. Ese tráfico en ráfagas congestiona las redes Ethernet tradicionales, produce atascos y retrasa a todo el clúster. No fueron diseñadas para IA.

## Conexión entre GPUs (dentro del servidor o rack)

- **NVLink (2016):** enlace directo entre GPUs, mucho más rápido que PCIe.
- **NVSwitch:** cuando las conexiones punto a punto ya no alcanzaban, nació un conmutador cuyo trabajo es que **cualquier GPU pueda hablar con cualquier otra**. Hasta 900 GB/s por GPU en la generación citada en el video.
- **Dominios de GPUs:** hasta 256 GPUs actuando como una sola, y con **NVLink Spine**, 576 GPUs en un único dominio con una memoria compartida descomunal (cifras del video).
- **SHARP:** protocolo que hace el **cálculo en tránsito**: la red va sumando o promediando los resultados mientras viajan, en lugar de esperar a que lleguen todos.

## Red entre servidores: Spectrum-X

- Ethernet rediseñado para IA, con switches de **51,2 terabits por segundo**.
- **Acelerador de red inteligente en cada servidor** (SuperNIC) y software de **enrutamiento adaptativo**: los paquetes esquivan las rutas congestionadas.
- **CPO (Co-Packaged Optics):** la óptica integrada junto al chip del switch, para gastar menos energía y mover más datos.

## Los demás componentes

- **DPU (unidad de procesamiento de datos) BlueField-3:** procesador dedicado a red, almacenamiento y seguridad, que libera a la CPU y la GPU de esas tareas.

- **Grace:** CPU de NVIDIA con arquitectura **Arm**, unida a la GPU con un enlace mucho más rápido que PCIe para superar el cuello de botella en el puente entre ambas.
- **Superchips:** Grace y GPU integrados en un solo módulo, una proeza de la ingeniería.
- **GB200 NVL72:** un rack con **72 GPUs Blackwell y 36 CPUs Grace** conectadas por NVLink, que funciona como una sola GPU gigante.
- **Blackwell:** arquitectura de GPU de 2024 con dos chips unidos como uno, Tensor Cores de nueva generación y precisiones bajas (FP4) para Transformers.

```
GPU → Superchip (Grace + GPU) → Servidor → Rack NVL72 (NVLink/NVSwitch)
                                                  ↓
                                    Clúster entre racks (Spectrum-X, BlueField-3)
```

# 4. Ejemplo para el tablero: entrenar una red sencilla

Una sola muestra, para poder hacer todas las cuentas a mano. Usamos vectores fila: $z = x \cdot W + b$.

## Planteamiento

```
Entrada x (2)  →  Capa oculta (2 neuronas, ReLU)  →  Salida ŷ (1, lineal)
```

- Entrada: **x = [1, 2]**. Valor deseado: **y = 1**.
- Pesos capa 1: **W1 = [[0,5 ; 1], [0,5 ; 0,5]]**, sesgo b1 = [0, 0].
- Pesos capa 2: **W2 = [[1], [0,5]]**, sesgo b2 = 0.
- Pérdida: **$L = \frac{1}{2}(\hat{y} - y)^2$**. Tasa de aprendizaje: **$\eta = 0{,}1$**.

## Paso 1: forward (hacia adelante)

```
z1 = x · W1 + b1 = [1·0,5 + 2·0,5 , 1·1 + 2·0,5] = [1,5 ; 2,0]
a1 = ReLU(z1) = [1,5 ; 2,0]            (ambos positivos, pasan igual)
ŷ  = a1 · W2 + b2 = 1,5·1 + 2,0·0,5 = 2,5
```

## Paso 2: pérdida

```
L = ½ (2,5 − 1)² = ½ · 2,25 = 1,125
```

La red predijo 2,5 y debía dar 1.

## Paso 3: backward (retropropagación, regla de la cadena)

Se va de atrás hacia adelante:

```
δ2 = dL/dŷ = ŷ − y = 1,5

dW2 = a1ᵀ · δ2 = [1,5·1,5 ; 2,0·1,5] = [2,25 ; 3,0]
db2 = δ2 = 1,5

δ1 = (δ2 · W2ᵀ) ⊙ ReLU'(z1)
   = [1,5·1 , 1,5·0,5] ⊙ [1 , 1] = [1,5 ; 0,75]

dW1 = xᵀ · δ1 = [[1·1,5 , 1·0,75],
                 [2·1,5 , 2·0,75]] = [[1,5 ; 0,75], [3,0 ; 1,5]]
db1 = δ1 = [1,5 ; 0,75]
```

(ReLU′ vale 1 donde z > 0 y 0 donde z < 0; aquí ambas neuronas están activas.)

## Paso 4: actualizar con descenso por gradiente

```
w ← w − η · gradiente

W2 = [1 − 0,1·2,25 ; 0,5 − 0,1·3,0] = [0,775 ; 0,2]
b2 = 0 − 0,1·1,5 = −0,15
W1 = [[0,5 − 0,15 , 1 − 0,075], [0,5 − 0,3 , 0,5 − 0,15]]
   = [[0,35 ; 0,925], [0,2 ; 0,35]]
b1 = [−0,15 ; −0,075]
```

## Paso 5: comprobar que mejoró

```
z1 = [1·0,35 + 2·0,2 − 0,15 , 1·0,925 + 2·0,35 − 0,075] = [0,6 ; 1,55]
a1 = [0,6 ; 1,55]
ŷ  = 0,6·0,775 + 1,55·0,2 − 0,15 = 0,625
L  = ½ (0,625 − 1)² ≈ 0,0703
```

La pérdida bajó de **1,125 a 0,0703** con un solo paso. Entrenar es repetir los pasos 1 a 4 miles o millones de veces. La caída es tan grande porque es una sola muestra y la tasa de aprendizaje es alta, y en una red real el progreso es mucho más gradual.

## ¿Dónde entra la GPU?

En el ejemplo había 1 muestra. En la realidad se procesa un **lote (batch)** de B muestras, y cada paso se convierte en una multiplicación de matrices:

| Paso | Con 1 muestra | Con un lote de B muestras |
|---|---|---|
| Forward | x (1×2) · W1 (2×2) | X (B×2) · W1 (2×2) |

| Paso | Con 1 muestra | Con un lote de B muestras |
|---|---|---|
| Gradiente de W1 | $x^T$ (2×1) · δ1 (1×2) | $X^T$ (2×B) · Δ1 (B×2) |

**Escala real:** una capa de 4096 → 4096 con un lote de 1024 muestras hace 2 · 1024 · 4096 · 4096 ≈ **34 mil millones de operaciones** solo en el forward. El backward cuesta cerca del doble, así que un paso completo ronda el triple.

La salida tiene 1024 × 4096 ≈ **4,2 millones de elementos**, y cada uno es un producto punto independiente de 4096 multiplicaciones. Una CPU los hace por tandas de unos pocos, mientras que una GPU asigna miles de hilos para trabajar al mismo tiempo.

**Cómo ordenar el tablero:** columna 1 con el diagrama de la red y los valores iniciales, columna 2 con forward y pérdida, columna 3 con backward y columna 4 con la actualización, la comprobación y la tabla de la GPU.

## 5. Código creativo: la biblioteca de los 4096 lectores

**Idea:** en el ejemplo de atención, cada palabra tenía que «consultar» a todas las demás. Imaginen una biblioteca donde cada libro debe leer a todos los demás para entender su contexto. Con 3 libros se hace a mano, con 4096 libros hace falta una multitud de lectores trabajando en paralelo. El código resuelve primero «El gato duerme» y luego pone a competir a la CPU y a la GPU con 4096 palabras.

```
import torch, time

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Lectores disponibles en:', device)

def atencion(Q, K, V):
    d_k = Q.shape[-1]
    pesos = torch.softmax(Q @ K.T / d_k ** 0.5, dim=-1)   # quién mira a
quién
    return pesos @ V, pesos                                # mezcla de
información

# --- Parte 1: la frase «El gato duerme» (los números de la clase) ---
Q = torch.tensor([[1., 0], [1, 2], [0, 2]], device=device)
K = torch.tensor([[1., 0], [0, 2], [1, 1]], device=device)
V = K.clone()
salida, pesos = atencion(Q, K, V)
print('Atención (filas: El, gato, duerme):')
print(pesos.cpu().round(decimals=2))
print('Vectores con contexto:')
print(salida.cpu().round(decimals=2))
```

```
# --- Parte 2: la carrera con 4096 palabras ---
def carrera(dev, n=4096, d=64, veces=10):
    Q, K, V = (torch.randn(n, d, device=dev) for _ in range(3))
    atencion(Q, K, V)                                    # calentamiento
    if dev == 'cuda': torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(veces):
        atencion(Q, K, V)
    if dev == 'cuda': torch.cuda.synchronize()
    return (time.perf_counter() - t0) / veces

t_cpu = carrera('cpu')
print(f'CPU: {t_cpu * 1000:.1f} ms por atención')
if torch.cuda.is_available():
    t_gpu = carrera('cuda')
    print(f'GPU: {t_gpu * 1000:.1f} ms por atención')
    print(f'La GPU fue {t_cpu / t_gpu:.0f} veces más rápida')
```

### Qué debería salir

- La matriz de atención de la parte 1 es aproximadamente `[[0,40 ; 0,20 ; 0,40], [0,07 ; 0,62 ; 0,31], [0,05 ; 0,77 ; 0,19]]`, igual que en los cálculos hechos a mano. El vector de «duerme» queda en `[0,23 ; 1,72]`.
- En la parte 2 la GPU suele ganar por un factor de decenas, pero depende del hardware (en Colab con una T4 se nota claramente).

### Cómo explicarlo en la exposición

1. `Q @ K.T` es la tabla de «quién mira a quién»: una multiplicación de matrices (de 3×3 en la parte 1 y de 4096×4096 en la parte 2).
2. `softmax` la convierte en porcentajes, y `pesos @ V` mezcla la información.
3. `device=dev` es lo único que cambia entre CPU y GPU: el código y el resultado son los mismos, solo cambia el tiempo.
4. `torch.cuda.synchronize()` espera a que la GPU termine, porque trabaja de forma asíncrona.
5. **Para ir más lejos:** envolver la llamada en `torch.autocast('cuda', dtype=torch.float16)` activa los Tensor Cores.

# Cierre

La historia en una línea: *la GPU nació para dibujar gráficos, resultó ideal para multiplicar matrices, y los Transformers la convirtieron en la pieza central de la IA, hasta el punto de que NVIDIA ya no construye solo chips sino racks y redes completas.*